

Gradual Typing

An Annotated Bibliography

Kei Rockwell (krockw@umich.edu)
University of Michigan, Ann Arbor

April 16, 2026

Both static and dynamic type systems have become a staple of programming language designs, each with their own benefits. Dynamic type systems in particular are convenient to work with early in a project’s lifecycle, allowing for rapid prototyping and easier adaptation to changing requirements. On the flip side, static typing can provide compile time soundness guarantees aiding maintainability, while also offering the potential for type-specific optimizations to be applied. This suggests that it is generally desirable to begin work on a project with a dynamically typed language and transition to static typing once the structure settles. However, rewriting a project simply to take advantage of a static type system would, in general, be an impractical proposition. This motivates type theories and type systems that allow for *convenient* evolution between typed and untyped code.

To address this, Siek and Taha introduced the idea of gradual typing [3]. At its core, this concept seeks to allow interpolation between a dynamic and statically typed language while providing as many static guarantees as possible. Siek and Taha cite prior work on Quasi Static types by Thatte as the starting point for their theory — this previous type system allowed for both static and dynamic typing within a program, but failed to catch certain type errors even in fully typed programs. They provided a gradually typed lambda calculus (GTLC) as a way to evolve from the dynamically typed lambda calculus to the simply typed lambda calculus (STLC). Crucially, they required that programs with complete type annotations, the GTLC should have identical static and dynamic semantics to the STLC. Similarly, for fully unannotated programs, if constants and other inherently typed expressions were cast to the dynamic type to avoid static type errors, they demand that the GTLC have the same semantics as the dynamic lambda

calculus. This work, however, did not seek to extend this to practical languages.

One drawback of the original proposal for gradual types is the deep network of casts that can arise when objects are passed between typed and untyped code repeatedly. This has the potential to cause significant performance slowdowns (due to space storing these casts) and to obscure the root cause of errors, hindering programmer experience. This is addressed by combining existing work on blame tracking [7] and merging casts to create a theory that allows for more performant runtime behavior while still providing precise location of errors for programmers.

From its introduction, there were numerous applications of the ideas behind to more mainstream programming languages, including Typed Racket, Reticulated (a gradual system for python), C#'s dynamic features, several iterations of safe typescript, and many others. However, the “convenient evolution” ideal was believed to be lost, as several practical implementations threw unexpected errors or were overly permissive when adding or removing type annotations. In response, Siek et al. introduced the gradual guarantee to formalize requirements for the gradual addition or removal of types [4]. This guarantee builds on the previous work of [3] and blame tracking work of [7] and others by demanding that changing the precision of type annotations within a program does not break these guarantees in a surprising manner. Many real-world implementations failed to satisfy this guarantee — some deliberately abandoned soundness, most prominently standard typescript — while others did so more organically. This work also highlighted likely performance challenges in projects implementing the gradual guarantee that would become a theme in the literature moving forward.

These performance challenges were further underlined a year later during profiling efforts on typed racket [5]. When benchmarking all precision levels of typing information, it was found that typed racket suffered from significant performance challenges due to runtime checks needed to guarantee soundness.

In response, there were several efforts to improve runtime performance of gradually typed languages. These included leveraging better optimization techniques at runtime [1], improving the representation used for interacting between typed and untyped regions [6], and reworking the type system (and overall language) to support faster run time type information [2]. Each of these works were able to significantly reduce the overhead of the gradual type system, albeit with some limitations both in performance and based on the underlying language design.

There are several avenues for future work on gradual typing. The first

continues this performance angle: even solutions that vastly improve performance still have potentially unacceptable overhead in some cases. Best case slowdowns for partially typed code in these benchmarks still hovers in the 10-15% range, which is not a cost that can be accepted in many applications. As a result, many languages designed to be gradually typed intentionally forgo runtime checks to guarantee soundness, missing out on a very useful portion of guarantees that should be provided by a type system.

Another avenue for future work on gradual typing focuses on broadening the understanding of the type systems that can be made gradual. The GTLC that provides the preeminent example of a gradually typed language builds on the type system of the *simply* typed lambda calculus, which is a very limited type system. Ongoing work focuses on extending the ideas behind gradual types in ways that enrich this type system while satisfying the gradual guarantee: for example, there have been several recent papers with the aim of creating a gradual dependent type system.

1 Project Proposal

For the project, I plan to implement a gradual type system for ALFA. This is in the spirit of the GTLC introduced in [3]. To evaluate this proposal, a reasonable first step seems to be testing the implementation against a static ALFA implementation to gain confidence that it catches static errors in fully typed programs. Extending this test suite with gradually typed ALFA programs for sanity checks would also be involved. Further evaluation seems challenging to explore in the timeframe, but I am of course open to feedback on this plan.

1.1 Note about annotations

Papers beyond the 6 paper requirement are cited, mostly due to relevance in the narrative. Full annotations for these papers are not provided. The papers with annotations are [1], [2], [3], [4], [5], and [6].

References

- [1] S. Bauman, C. F. Bolz-Tereick, J. Siek, and S. Tobin-Hochstadt, “Sound gradual typing: only mostly dead,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, Oct. 2017. [Online]. Available: <https://doi.org/10.1145/3133878>

This paper pursues the potential for performance improvements by improving the JIT in use identified by Takikawa et al. as part of their Typed Racket benchmarks. They build on Pycket, a JIT developed using work originally for the PyPy implementation of Python. Furthermore, they adapt existing methodology from prototype-based class systems to create more efficient implementations of runtime casts and type checking.

To evaluate their work, the authors first run the benchmark suite of Takikawa et al. on the Typed Racket implementation and using Pycket. They found that Pycket significantly reduced the overhead of gradual typing, in some cases from 100x to below 2x. 45% of configurations resulted in only a 10% slowdown, and 60% of the test instances are *close* (in the sense of requiring few increases in the amount of type information) to an instance that only has a 10% slowdown. When with the improvement in baseline execution speed, these improvements make the overhead of gradual typing much more acceptable. Furthermore, the authors identified key areas where Pycket created performance speedups: better hot loop detection, even in the presence of objects that obscure the actual behavior of the program due to runtime type checking. This allows for better inlining decisions on specific runtime checks, as well as additional checks being optimized away entirely due to being unnecessary.

While Pycket enabled significant performance gains, the authors stress that this is not the last word in the area. The overhead is still too high for some applications, and Typed Racket is a language that by design limits the variance in typing to either have fully typed or fully untyped modules. This design limits interaction between untyped and typed code more than other languages. Furthermore, they emphasize that Pycket’s performance gains can largely be traced to the fact that it is a tracing JIT: extending these results to less complex JITs is still an open problem.

- [2] F. Muehlboeck and R. Tate, “Sound gradual typing is nominally alive and well,” *Proc. ACM Program. Lang.*, vol. 1, no. OOPSLA, Oct. 2017. [Online]. Available: <https://doi.org/10.1145/3133880>

This paper addresses the performance concerns of gradually

typed languages identified by Takikawa et al. the year prior. They use several ingredients that differ from most existing implementations to address this: one is a nominal type system. They propose that a nominal type system allows for faster precise run time type information (RTTI) to minimize the cost of runtime checks. They also propose two other properties that a gradual language could satisfy to have better performance: transparency, requiring that casts are invisible after evaluation; and immediate accountability, requiring that typing problems must be immediately identified. Finally, they propose that implementing a language from the ground up to be gradually typed offers potential performance speedups compared to languages that add the overhead of dynamic typing onto an untyped language at runtime.

To evaluate these claims, the authors define a simple gradually typed nominal object oriented language called Nom. They show it satisfies both their new properties as well as the gradual guarantee of Siek et al. They implement a prototype compiler for Nom and evaluate its performance compared to other benchmarks of gradual languages, finding that the performance was much more consistent across varying typing precision compared to the previous typed Racket benchmarking efforts. Finally, they ported the same benchmarks to C# to compare Nom to another nominally typed language supporting gradual types: C# exhibited a much more pronounced slowdown using gradually typed code than Nom did.

The authors acknowledge that their benchmarking covered a relatively small set of programs, likely as a result of being unable to directly use existing benchmarks from another language. They also highlight that the type system implemented in Nom is quite limited compared to the structural systems used by most other gradually typed language, underlining this as a direction for future work. Furthermore, they note a need for further theoretical work to reconcile gradual typing and to generics for nominal type systems. Finally, they observe that implementing first class functions is likely a future performance headache.

- [3] J. G. Siek and W. Taha, “Gradual typing for functional languages,” in *Scheme and functional programming workshop*, vol. 6, 2006, pp. 81–92.

This paper introduces the concept of gradual typing. The problems addressed are twofold: firstly, as noted in the narrative summary, it is beneficial to gradually transition from dynamic to static typing. Secondly, while there are systems that seek to allow both dynamic and static types in a single program, they either explicitly did not attempt to give static guarantees or were too weak in the guarantees that they offered. This paper introduces a Gradually Typed Lambda Calculus (GTLC) as an example gradually typed language that both offers much stronger static guarantees and has runtime checks at boundaries between untyped and typed code to guarantee soundness. It offers both static and dynamic semantics for the GTLC.

To evaluate this work, the authors offer mechanized proofs that the GTLC is both type safe and also demonstrate that it correctly catches the motivating examples of programs that should be rejected statically but were not by previous type systems. They also reason about convenient evolution by designing the GTLC to be equivalent to the simply typed lambda calculus when a program is fully annotated, and equivalent to a dynamically typed lambda calculus (up to adding casts to the unknown type in some places) in a program with no type annotations.

The authors leave open richer type systems, including support for object oriented programming paradigms, and implementing this work in other languages. They also mention performing studies to identify the impact on programmer productivity.

- [4] J. G. Siek, M. M. Vitousek, M. Cimini, and J. T. Boyland, “Refined criteria for gradual typing,” in *1st Summit on Advances in Programming Languages (SNAPL 2015)*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, 2015, pp. 274–293.

This paper identifies a dilution of the term “gradual typing” in the literature, noting that many works on the subject no longer consider the convenient evolution from untyped to typed code, instead focusing simply on difficulties integrating dynamic and static type systems. The authors believe this is partially due to a failure of previous work to formalize the requirements of “convenient evolution.” They resolve this problem by in-

roducing the gradual guarantee: informally, removing type annotations should not impact the behavior of the program, while adding *correct* type annotations should also not impact the behavior.

To evaluate the gradual guarantee, the authors begin with a mechanized proof that the GTLC meets this gradual guarantee, demonstrating that this guarantee did indeed match the original blueprint for gradual typing and could potentially be implemented. They also evaluated several other gradually typed languages against the gradual guarantee: these results were generally less positive. They did not prove any practical language met the gradual guarantee, and indeed showed that many examples in the literature did not.

This work identified difficulty in proving the gradual guarantee for more complicated languages, and identifies investigating if it could be proven on such a language as a direction for future work. Furthermore, they note that even languages which seem to satisfy the guarantee had not explicitly accounted for the evolution of code, which was the main motivating point behind gradual typing. More thoroughly accounting for this evolution in language design, with or without proving the gradual guarantee, was also identified as necessary future work.

- [5] A. Takikawa, D. Feltey, B. Greenman, M. S. New, J. Vitek, and M. Felleisen, “Is sound gradual typing dead?” in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 456–468. [Online]. Available: <https://doi.org/10.1145/2837614.2837630>

This paper identifies a general lack of benchmarking in the gradual typing literature. Furthermore, when benchmarking is considered, it both tended to lack coverage across different levels of typing precision and already indicate poor performance. To address this, the authors create a benchmarking framework for Typed Racket that tests the performance of relatively small (but user-written) programs at all possible levels of typing precision.

This benchmarking found generally very poor performance metrics for Typed Racket, with almost all examples tested dis-

playing at *minimum* a 3x slowdown. Some examples, particularly the more complicated ones, exhibited slowdowns over 100x. Over 80% of all time spent executing partially typed programs was in the runtime type checking needed to ensure soundness. These programs were also generally far away from the (slightly) more manageable slowdowns: many instances exhibiting over 10x slowdowns required adding significantly more type annotations before the penalty for partial typing was under 10x.

The authors identified several limitations: firstly, this benchmark was only run on Typed Racket. They conjecture that it is possible (but unlikely) that a different language could avoid most of these slowdowns. Furthermore, they note that the implementation of Typed Racket is somewhat unoptimized, and conjecture that a better Just In Time compiler (JIT) that can apply more profiling based optimizations could potentially minimize the overhead by eliminating checks. Finally, they observe that Typed Racket (and other gradually typed languages) had seen some use in practice despite these performance concerns, and conjecture that programmer practice might somehow reduce the overhead to acceptable levels for these applications.

- [6] M. M. Vitousek, C. Swords, and J. G. Siek, “Big types in little runtime: open-world soundness and collaborative blame for gradual type systems,” *SIGPLAN Not.*, vol. 52, no. 1, p. 762–774, Jan. 2017. [Online]. Available: <https://doi.org/10.1145/3093333.3009849>

This paper identifies the trend of implementing gradually typed languages on top of an underlying untyped language: for example, typescript on top of javascript. In this setting, it is often desirable to allow for existing libraries written in the untyped language to be interacted with by the typed language. However, ensuring soundness in this setting is challenging to do efficiently, and some examples do not bother (e.g. typescript assumes correctly typed values will be provided). Even if checks are inserted, the authors identify most proofs regarding soundness assume all code originates from the gradually typed language — a key oversight when interaction with libraries is a design requirement.

The authors first address this hole in the theory by formalizing an open world soundness property that requires code be sound even when interacting with code not originating from gradually typed code. They then propose a new “transient” approach to implementing gradually typed languages that makes implementing a sound translation to the untyped language easier. They define formal static, dynamic, and blame semantics for this new approach.

To evaluate this work, the authors prove that a variant of the lambda calculus using this new approach meets their new, stronger soundness guarantee. They also prove it satisfies the gradual guarantee of Siek et al. that presented a stricter requirement for gradual languages. Finally, they prove that their new blame system for this transient approach satisfies correctness guarantees similar to those in prior work.

In light of the benchmarking efforts of Takikawa et al. from the previous year, the authors also implemented this approach for Reticulated Python, a gradual typing system for Python. They conjecture (notably without proof) that this implementation of Reticulated Python is open world sound based on a test suite developed by one of the author’s prior work. They run this implementation a selection of programs from the Python 3.4 benchmark suite, comparing the performance to unmodified Python 3.4. They found that with blame tracking disabled (but still using runtime checks for soundness), the overhead was at worst 5.4x slower, and in many cases several times better than that. The authors note that this is well within acceptable performance for many use cases of Python. Furthermore, while the performance with blame tracking is up to 18x slower, they note that this could still be acceptable when debugging, which is the primary use case of blame tracking.

There are several limitations to the work of this paper. Firstly, the reticulated python benchmarking was at one level of annotation precision, unlike previous work on Typed Racket. Furthermore, the benchmarks selected did not extensive use of external libraries so as to not obscure the performance characteristics with time spent in library code. However, this misses a large class of python programs. Finally, the new guarantees introduced in this paper were not proven for

any practical language, and in particular not for the Reticulated Python implementation used for benchmarking.

- [7] P. Wadler and R. B. Findler, “Well-typed programs can’t be blamed,” in *European Symposium on Programming*. Springer, 2009, pp. 1–16.